

Claude Code: Ready Reference Guide

A quick-reference for features, usage patterns, and best practices.

1. What Claude Code Is

Claude Code is an agentic coding environment, not a chatbot. It reads files, runs commands, edits code, and works through multi-step tasks autonomously while you watch, redirect, or step away. The core shift: instead of writing code and asking for review, you describe the outcome and Claude explores, plans, and implements.

2. Core Features

File and code operations

- Read, create, edit, and overwrite files; can read multiple files at once.
- Multi-file edits in a single instruction (e.g., interface changes across related files).
- Generate code from natural language: functions, components, API endpoints, config files.
- Refactor: restructure code, rename symbols, apply patterns while preserving behavior.
- Debug: analyze error messages/stack traces and implement fixes.
- Review diffs for bugs, security issues, performance problems, and style violations.

Project understanding

- Search by filename, content, and regex.

- Analyze import graphs, dependency trees, detect circular dependencies.
- Explain unfamiliar code, ask "why does X call Y instead of Z" style questions.

Terminal and processes

- Run shell commands, manage background processes, check ports.
- `cat error.log | claude` pipes data directly into a session.

Sessions and history

- Conversations persist locally — `claude --continue` resumes the latest, `claude --resume` picks from a list.
- `/rename` to name sessions like git branches (e.g., `oauth-migration`).
- `/recap` — one-line summary of what happened, shown automatically on return.
- `/btw` — ask a side question that doesn't pollute the main context; answer shows in a dismissible overlay.

Checkpoints and rewind

- Every prompt creates a checkpoint (auto file snapshots before changes).
- `Esc` — interrupt mid-action, context preserved.
- `Esc + Esc` or `/rewind` — restore conversation, code, or both to an earlier point, or summarize from there.
- Checkpoints persist across sessions but are **not** a git replacement (only track Claude's changes).

Context management

- `/clear` — reset context between unrelated tasks.

- `/compact <instructions>` — manual compaction with focus, e.g. `/compact Focus on the API changes`.
- Auto-compaction triggers near context limits, preserving key code/decisions.

Extension mechanisms

Mechanism	Purpose	Adherence
CLAUDE.md	Persistent project context (commands, style, workflow)	Advisory (~80%)
Hooks	Scripts that run automatically at workflow points (lint, format, security checks)	Deterministic (100%)
Skills	<code>SKILL.md</code> files for domain knowledge / reusable workflows, loaded on demand	Auto-applied or <code>/skill-name</code>
Subagents	Specialized assistants in <code>.claude/agents/</code> , own context + tool scope	Invoked explicitly or delegated
Plugins	Bundles of skills/hooks/subagents/MCP servers from marketplace (<code>/plugin</code>)	Installed
MCP servers	Connect external tools (DB, Notion, Figma, issue trackers) via <code>claude mcp add</code>	Tool-call based

Permission modes

- **Default:** prompts for file writes, bash commands, MCP tool calls.
- **Auto mode** (`--permission-mode auto`): a classifier blocks only risky actions (scope escalation, unknown infra); routine work proceeds without prompts.

- **Permission allowlists** (`/permissions`): pre-approve specific safe commands (`npm run lint`, `git commit`).
- **Sandboxing** (`/sandbox`): OS-level filesystem/network isolation.

Automation and scale

- **Non-interactive mode:** `claude -p "prompt"` for CI, pre-commit hooks, scripts. Supports `--output-format json` / `stream-json --verbose`.
- **Parallel sessions:** git worktrees, Desktop app, Claude Code on the web (cloud VMs), or agent teams for coordinated multi-session work.
- **Fan-out:** loop `claude -p` over a file list with `--allowedTools` to scope permissions for batch jobs (e.g., large migrations).

Output styles

- `/config` to pick Explanatory, Concise, or Technical response styles, or define custom styles in `~/.claude/output-styles/`.
-

3. Best Practice Workflows

3.1 Explore → Plan → Implement → Commit

1. **Explore** (plan mode): Claude reads files and answers questions without editing.
2. **Plan:** ask for a detailed implementation plan; edit it directly with `Ctrl+G`.
3. **Implement:** exit plan mode, let Claude code and verify against the plan.
4. **Commit:** ask Claude to write a descriptive commit message and open a PR.

Skip planning for small, clearly-scoped changes (typo fixes, single-variable renames) — if you can describe the diff in one sentence, just do it directly.

3.2 Give Claude a way to verify its work

Without a pass/fail signal, "looks done" is the only stopping condition. Always provide one of:

- A test suite or specific test cases to run.
- A build/lint command with an exit code.
- A screenshot to compare UI output against.
- A `/goal` condition (re-checked every turn until it holds).
- A Stop hook that gates session end on a script passing.

Ask Claude to **show evidence** (test output, command + result, screenshots) rather than just asserting success.

3.3 Write specific prompts

Instead of	Try
"add tests for foo.py"	"write a test for foo.py covering the logged-out edge case; avoid mocks"
"add a calendar widget"	"follow the pattern in HotDogWidget.php to build a calendar widget with month select + year pagination, using only existing libraries"
"fix the login bug"	"users report login fails after session timeout — check token refresh in src/auth/, write a failing test that reproduces it, then fix"

Use `@file` references, paste screenshots/images directly, give doc URLs (allowlist via `/permissions`), or pipe data with `cat file | claude`.

3.4 Let Claude interview you for big features

For larger work, start minimal and have Claude use `AskUserQuestion` to interview you on implementation, UI/UX, edge cases, and tradeoffs — then write a self-contained `SPEC.md` (named files/interfaces, scope boundaries, an end-to-end verification step). Start a **fresh session** to implement from the spec.

3.5 Use subagents for investigation and review

- Delegate research: `"use subagents to investigate how auth handles token refresh"` — runs in a separate context, returns a summary, keeps your main session clean.
- Use a fresh-context subagent (or `/code-review`) for an **adversarial review** of a diff against a plan before calling work done — it evaluates the result without the implementer's reasoning bias.
- Writer/Reviewer pattern across two sessions: one implements, the other reviews with fresh eyes, feedback flows back.

3.6 Configure your environment once

- `/init` to generate a starter `CLAUDE.md` from your project structure.
- Keep CLAUDE.md short: bash commands Claude can't guess, non-default style rules, test runners, repo etiquette, architectural decisions, env quirks, gotchas. Exclude anything Claude can infer from the code itself.
- Use `@path/to/file` imports inside CLAUDE.md to pull in README, package.json, or personal override files.
- Locations: `~/ .claude/CLAUDE.md` (global), `./CLAUDE.md` (shared, commit to git), `./CLAUDE.local.md` (personal, gitignore), plus parent/child directory files for monorepos.
- Use "IMPORTANT" / "YOU MUST" emphasis sparingly to boost adherence on critical rules.

- Move anything that must happen with **zero exceptions** (formatting, linting, security checks) into a **hook** instead of CLAUDE.md.
-

4. Common Failure Patterns and Fixes

Pattern	Symptom	Fix
Kitchen sink session	Mixing unrelated tasks in one context	<code>/clear</code> between tasks
Repeated correction	Same mistake corrected 2+ times	<code>/clear</code> , write a better prompt incorporating what you learned
Over-specified CLAUDE.md	Claude ignores half the rules	Ruthlessly prune; convert hard rules to hooks
Trust-then-verify gap	Plausible code with broken edge cases	Always require tests/scripts/screenshots before shipping
Infinite exploration	Hundreds of files read, context exhausted	Scope investigations narrowly; delegate to subagents

5. Quick Command Reference

Command	Purpose
<code>/init</code>	Generate starter CLAUDE.md

Command	Purpose
<code>/config</code>	Set output style (Explanatory/Concise/Technical)
<code>/permissions</code>	Manage allowlists and domain access
<code>/sandbox</code>	Toggle OS-level isolation
<code>/hooks</code>	Browse configured hooks
<code>/clear</code>	Reset context
<code>/compact <instructions></code>	Manual context compaction
<code>/rewind</code>	Open checkpoint/rewind menu
<code>/rename</code>	Name the current session
<code>/recap</code>	Summarize what happened in the session
<code>/btw</code>	Side question, doesn't enter context
<code>/plugin</code>	Browse plugin marketplace
<code>claude -p "..."</code>	Non-interactive (CI/scripts) mode
<code>claude --continue</code> / <code>--resume</code>	Resume sessions

6. Suggested Use in a Portfolio Workflow

For structured, multi-assignment projects (e.g., a CLI tool → API/web app → refactor → MCP workflows):

- Use CLAUDE.md per project to encode build/test/lint commands and code style from day one — this becomes part of the artifact and signals engineering maturity to reviewers.
- Treat CI (lint + test across versions) as a hook-enforced or CLAUDE.md-documented expectation, written early, not bolted on.
- For new assignments with architectural decisions still open (e.g., frontend stack choice), use plan mode and the interview pattern to produce a `SPEC.md` before coding — this spec itself becomes a useful artifact for showing design thinking.
- Use subagent-based review on completed assignments before considering them "done" — catches gaps a self-review misses.